

SPIFFE + SPIRE mTLS Demo

File-by-file explanation of the spiffe-spire-mtls-demo repository

Project flow: Docker creates the environment -> SPIRE Server establishes the trust domain -> SPIRE Agent becomes trusted -> Agent attests workloads -> registration entries map selectors to SPIFFE IDs -> workloads retrieve X.509-SVIDs -> Node.js uses those credentials for mTLS.

1. docker-compose.yml - Wires the Lab Together

This is the orchestration file. It defines the SPIRE Server, SPIRE Agent, Client, and API containers and connects the pieces needed for workload identity.

It mounts the Server and Agent configuration, gives the Agent access to Docker metadata through `/var/run/docker.sock`, and shares the Agent Workload API socket with the Client and API. It also assigns the Docker workload labels used by the Docker WorkloadAttestor.

```
Client label: spiffe.workload: "client"
API label: spiffe.workload: "api"
```

Those labels matter because SPIRE turns observed workload properties into selectors. Changing the Client label to `broken-client` makes the observed selector stop matching the Client registration entry, so SPIRE refuses to issue the Client identity.

Mental shortcut: `docker-compose.yml` creates and connects the environment; it does not itself grant SPIFFE identities.

2. spire/server/server.conf - Configures the Identity Authority

This configures the SPIRE Server, the central authority for the demo. It establishes `demo.local` as the trust domain, listens for Agents, configures SPIRE state storage, enables join-token node attestation, and configures management of the Server's cryptographic keys.

```
trust_domain = "demo.local"

spiffe://demo.local/client
spiffe://demo.local/api
```

The join-token NodeAttestor is why the demo generates a one-time token and supplies it when bootstrapping the Agent. That process establishes trust in the Agent before it can participate in workload identity issuance.

Mental shortcut: `server.conf` answers: Who is the identity authority? What trust domain does it control? How can an Agent join it? How are Server state and keys managed?

3. spire/agent/agent.conf - Configures the Workload Identity Broker

This configures the SPIRE Agent. It tells the Agent which trust domain it belongs to, where the SPIRE Server is, where to expose the local Workload API, and which plugins to use for node and workload attestation.

```
/run/spire/sockets/api.sock
```

The Docker WorkloadAttestor is especially important. It lets the Agent inspect Docker metadata and derive selectors such as:

```
docker:label:spiffe.workload:client
docker:label:spiffe.workload:api
```

SPIRE then compares those observed selectors with registration entries. A matching registration determines which SPIFFE ID the workload is eligible to receive.

Mental shortcut: agent.conf answers: How does the Agent join SPIRE? How does it identify workloads? Where do workloads request identity?

4. Registration Entries - The Policy Mapping

Registration entries are created with SPIRE commands rather than stored as repository files in this demo, but they are essential to understanding the file flow. They map attested workload selectors to SPIFFE IDs under the currently attested Agent.

```
docker:label:spiffe.workload:client -> spiffe://demo.local/client
docker:label:spiffe.workload:api -> spiffe://demo.local/api
```

The workload does not ask SPIRE for a specific identity. The Agent identifies the caller, derives selectors, and SPIRE uses registration policy to determine the identity.

5. api/Dockerfile - Builds the API Workload

This defines the API container image. Its SPIFFE-related responsibility is to ensure the container has the SPIRE Agent CLI used by the entrypoint to fetch credentials before the Node.js API starts.

```
Dockerfile -> entrypoint.sh -> obtain SPIFFE credentials -> Node API starts
```

The Dockerfile does not assign `spiffe://demo.local/api`. The identity is determined through workload attestation and registration policy.

6. api/entrypoint.sh - Retrieves the API Identity

This script is the bridge between SPIRE identity issuance and the API application. Before Node starts, it contacts the Agent's Workload API and exports the API's credentials into `/tmp`.

```
/tmp/svid.0.pem  
/tmp/svid.0.key  
/tmp/bundle.0.pem
```

These are the X.509-SVID certificate, corresponding private key, and trust bundle. Because the script uses `set -e`, identity retrieval failure stops startup immediately. The application never starts without the required SPIFFE credentials.

Mental shortcut: Get the SPIFFE credentials first. If SPIRE refuses identity issuance, do not start the application.

7. api/src/index.ts - Uses the API Identity for TLS

This is the Node.js/Express API. It reads the certificate, private key, and trust bundle exported by the entrypoint and supplies them to the HTTPS server. The API presents its SPIRE-issued certificate and requires a client certificate that chains to the configured trust bundle.

```
SPIRE issues identity  
|  
entrypoint.sh retrieves identity  
|  
index.ts uses identity cryptographically
```

Important limitation: the demo performs certificate trust validation but does not implement an explicit application-level authorization rule requiring the peer URI SAN to equal exactly `spiffe://demo.local/client`.

8. api/package.json, package-lock.json, tsconfig.json

package.json defines Node dependencies and scripts. **package-lock.json** pins the exact dependency graph. **tsconfig.json** controls TypeScript compilation. These support the API application but do not determine SPIFFE identity.

9. client/Dockerfile - Builds the Client Workload

This creates the Client runtime and includes the SPIRE credential-fetching capability used before the Node client starts. Like the API Dockerfile, it does not assign the Client SPIFFE ID.

10. client/entrypoint.sh - Retrieves the Client Identity

The Client entrypoint contacts the same local Workload API socket and asks for an X.509-SVID. Crucially, the Client does not request `spiffe://demo.local/client` by name.

```
Client contacts Workload API
|
Agent determines the caller
|
Workload attestors derive selectors
|
Selectors match registration policy
|
SPIRE determines the SPIFFE ID
|
X.509-SVID is returned
```

Key concept: The workload does not choose or claim its own SPIFFE ID. Identity follows from attestation evidence and registration policy.

11. client/src/index.ts - Initiates the mTLS Request

This is the Client application. It reads its X.509-SVID, private key, and trust bundle and uses them to initiate an HTTPS request to the API.

```
Client X.509-SVID + private key + trust bundle
|
HTTPS / mTLS
|
API :3000
```

When the selector and registration match, the Client receives its identity and the request succeeds. The current demo disables conventional hostname checking and does not perform an exact SPIFFE-ID match against `spiffe://demo.local/api`, so it demonstrates trust-chain validation rather than complete SPIFFE-aware peer authorization.

12. client/package.json, package-lock.json, tsconfig.json

These have the same support roles as the API equivalents: dependencies and scripts, exact dependency versions, and TypeScript compiler configuration. They are application-support files rather than identity-policy files.

13. README.md - Documents the Architecture

The README explains the purpose of the demo, the `demo.local` trust domain, the Client and API SPIFFE IDs, and the high-level architecture. It documents the system but does not participate in runtime identity issuance.

14. .gitignore - Keeps Repository Noise Out of Git

This prevents generated or local artifacts from being committed. It has no direct role in SPIFFE/SPIRE identity.

Complete File-to-Flow Mental Model

```
docker-compose.yml
|
Creates Server, Agent, Client, API
|
server.conf
|
SPIRE Server controls demo.local
and establishes Agent trust
|
agent.conf
|
Agent exposes Workload API and attests workloads
|
Docker labels + WorkloadAttestor
|
Agent derives workload selectors
|
Registration entries
|
client selector -> spiffe://demo.local/client
api selector -> spiffe://demo.local/api
|
client/entrypoint.sh + api/entrypoint.sh
|
Each workload retrieves X.509-SVID + key + trust bundle
|
client/src/index.ts + api/src/index.ts
|
Node.js uses the credentials for TLS
|
CLIENT ----- mTLS -----> API
```

The six responsibilities to memorize: docker-compose.yml creates the environment. server.conf defines the identity authority. agent.conf defines how the Agent joins that authority and attests workloads. Registration entries map observed workload properties to SPIFFE IDs. The entrypoint scripts retrieve the resulting identities. The index.ts files use the resulting cryptographic material for the connection.

Negative-test proof: correct selector -> registration match -> identity issued -> SVID -> application starts -> mTLS succeeds. Broken selector -> no registration match -> no identity -> no SVID -> application does not start -> mTLS cannot begin.